AI Gateway Baselines

Obvious Baselines for the AI Gateway Project

Several simple and intuitive baselines can be implemented for the AI Gateway system. These baselines provide reference points for evaluating more advanced routing approaches.

Naive Baselines

These methods do not attempt to understand the user query and instead route requests based on simple statistics or randomization.

- **Uniform Random Routing**: Requests are assigned uniformly at random among all available models that satisfy the predefined cost and latency constraints. This serves as the lowest-level baseline [1].
- **Round Robin Routing**: Incoming requests are distributed sequentially across the available models in a cyclic manner.
- **Static Extreme Selection** [2]:
  - **Always Cheapest**: Every request is routed to the lowest-cost model (e.g., GPT-5.2).
  - **Always Fastest**: Every request is routed to the model with the lowest average latency.
  - **Always Best**: Every request is routed to the highest-performing model (e.g., GPT-5.5 or Claude Opus), regardless of cost considerations.

Heuristic / Rule-Based Baselines

These methods use simple characteristics of the prompt to make routing decisions.

- **Prompt-Length-Based Routing** [4]:
  A simple rule commonly used as an initial gateway policy is to route short prompts to a fast and inexpensive model, while longer prompts are routed to a more capable model.
- **Keyword-Based Routing** [5]:
  Simple keyword matching techniques (e.g., regular expressions) can be used to identify potentially complex tasks. If the prompt contains keywords associated with reasoning, analysis, programming, mathematics, or similar domains, it is routed to a stronger model. Otherwise, general conversational queries are routed to a cheaper model.

Greedy Utility-Based Baseline

A straightforward mathematical baseline is to define a static utility function for each model.

$$U_i = w_1 Q_i - w_2 C_i - w_3 L_i \tag{1}$$

where:

- Q_i denotes the average quality of model i,
- C_i denotes its average cost,
- L_i denotes its average latency,
- w_1, w_2, and w_3 are fixed weighting coefficients.

The router selects the model with the highest utility score while satisfying the constraints

$$C_i \leq C_{\max} \tag{2}$$

and

$$L_i \leq L_{\max}. \tag{3}$$

This approach is inspired by utility-based decision frameworks such as Cost-of-Pass [3].

Cascade Routing

Cascade routing is a widely adopted industrial strategy [2]. A request is first sent to a cheap and fast model. Subsequently, a lightweight evaluator examines the generated response. The evaluator may use simple heuristics such as response length, presence of error indicators, or confidence signals. If the response is deemed insufficient, the request is escalated to a stronger and more expensive model.

This approach aims to achieve a favorable trade-off between cost and quality by reserving expensive model calls for only the most challenging queries.

References

[1] BerriAI: Litellm. `https://github.com/BerriAI/litellm` (2026)

[2] Chen, L., Zaharia, M., Zou, J.: Frugalgpt: How to use large language models while reducing cost and improving performance. Transactions on Machine Learning Research (2024)

[3] Erol, M.H., El, B., Suzgun, M., Yuksekgonul, M., Zou, J.: Cost-of-pass: An economic framework for evaluating language models. In: International Conference on Learning Representations (ICLR) (2026)

[4] Ong, I., et al.: Routellm: Learning to route llms from preference data. In: International Conference on Learning Representations (ICLR) (2025)

[5] Zuplo: Zuplo ai gateway documentation. `https://zuplo.com/docs/ai-gateway/introduction` (2026)